Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering and Research, Pune

Department of Computer Engineering

Subject: Operating System

Teaching Faculty:

Mrs. Shrutika A. Menkudale

Introduction to Syllabus

Course :- PCC-203- COM: Operating Systems

Teaching /scheme	Credits	Examination Scheme
Theory : 03 Hours/Week	03	CCE : 30 Marks End-Semester: 70 Marks

Prerequisite Courses : Data Structure, Digital Electronics

Companion Course : Computer Organization and Microprocessors

Course Objective

1. To **understand** the fundamental concepts, types, and structures of Operating Systems.

1. To **understand** and **analyze** process management concepts.

1. To **identify** and **solve** concurrency and deadlock in the operating system.

1. **Explore** the various techniques of memory management.

1. **Understand** I/O management, disk scheduling, and file systems

Course Outcome

- CO1: **Analyze** the fundamentals of Operating Systems, including types, structures, system calls, and basic Linux commands.

- CO2: **Apply** process scheduling and synchronization to optimize CPU utilization in modern operating systems.

- CO3: **Identify** the mechanism for dealing with deadlocks and concurrency concerns.

- CO4: **Apply** techniques of memory management to solve memory management problems.

- CO5: **Illustrate** I/O and file management policies.

Course Syllabus

Unit IV- Memory Management (09 Hours)

Introduction: Contiguous and non-contiguous.

Fragmentation: Internal and External fragmentation.

Memory allocation strategies: First Fit, Best Fit, Worst Fit, Memory **Partitioning:** Fixed and Dynamic partitions.

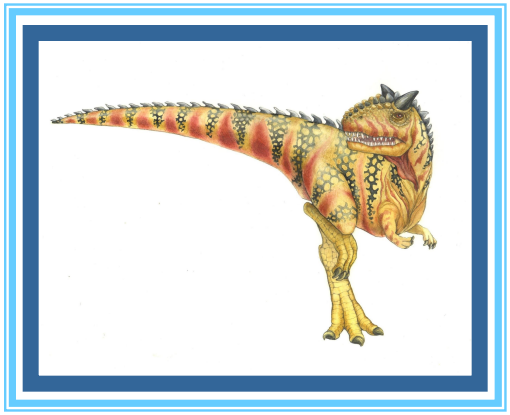
Paging: Structure of page table, Segmentation.Virtual Memory: Background, Demand Paging.

Page Replacement: FIFO, LRU, Optimal Thrashing.

Case study :

Intel Premium

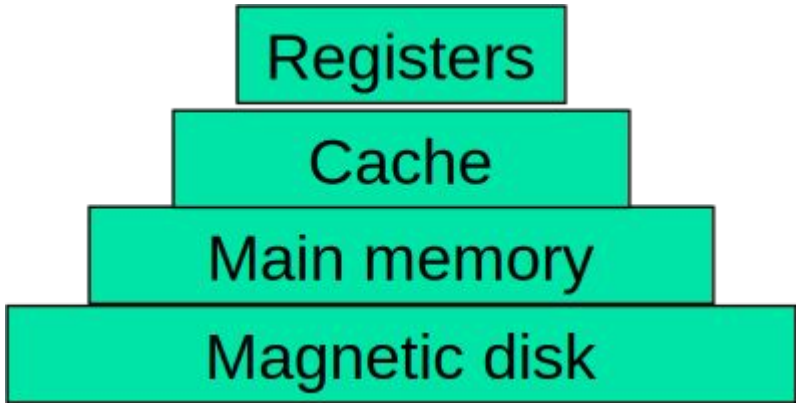
UNIT IV:
Memory Management



Silberschatz, Galvin and Gagne ©2018

Why Memory Management?

- Why money management?
- Not enough money. Same thing for memory
- Parkinson's law: programs expand to fill the memory available to hold them
- "640KB memory are enough for everyone" – Bill Gates
- Programmers' ideal: an infinitely large, infinitely fast memory, nonvolatile
- Reality: memory hierarchy

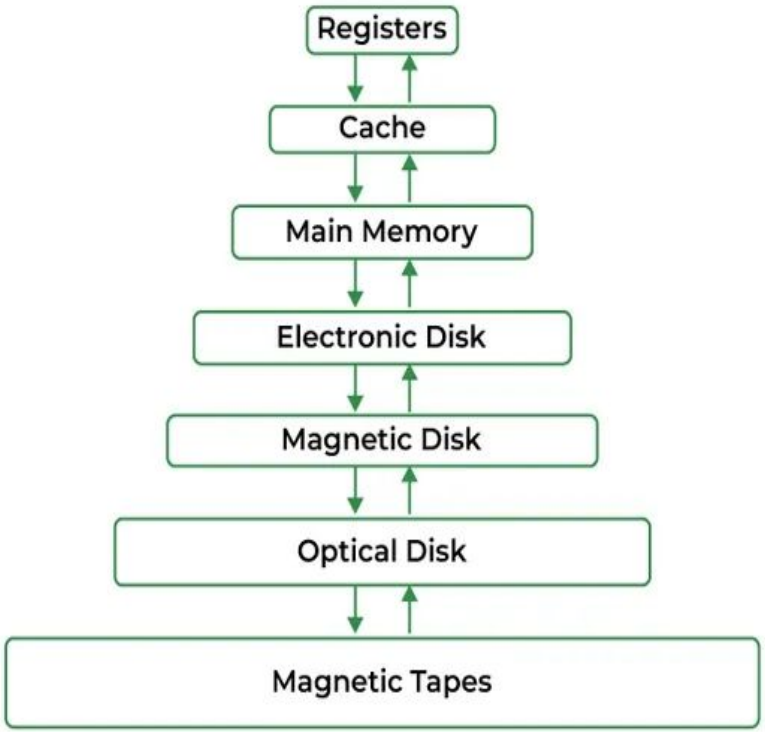


What Is Memory Management?

- Memory manager: the part of the OS managing the memory hierarchy
- Keep track of memory parts in use/not in use
- Allocate/de-allocate memory to processes
- Manage swapping between main memory and disk
- Basic memory management: every program is put and run in main memory as whole
- Swapping & paging: move processes back and forth between main memory and disk

Memory Management

- Memory management is a critical aspect of operating systems that ensures efficient use of the computer's memory resources. It controls how memory is allocated and deallocated to processes, which is key to both performance and stability. Below is a detailed overview of the various components and techniques involved in memory management.

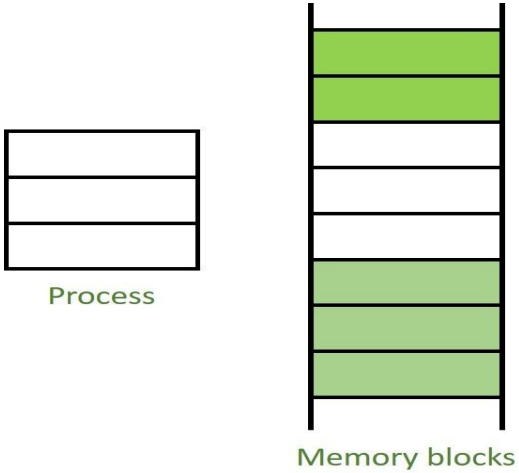


Memory Management

- **Registers –** Fastest memory, located inside the CPU, holds data currently being processed.
- **Cache –** Very fast memory close to CPU, stores frequently accessed data to reduce access time.
- **Main Memory (RAM) –** Primary storage for running programs and data; faster than disks but slower than cache.
- **Electronic Disk (SSD) –** Faster than traditional disks, used for secondary storage.
- **Magnetic Disk (HDD) –** Slower but large-capacity storage for data and programs.
- **Optical Disk (CD/DVD) –** Removable storage, slower than magnetic disks.
- **Magnetic Tapes –** Very slow, used for archival and backup because of low cost and high capacity.

Introduction: Contiguous and non-contiguous.

- **What is Contiguous Memory Allocation?**
- Contiguous memory allocation is basically a method in which a **single contiguous section/part of memory is allocated to a process or file needing it.**
- Because of this, all the available memory space resides in the same place together, which means that the freely/unused available memory partitions are not distributed in a random fashion here and there across the whole memory space.



Contiguous Memory Allocation

Introduction: Contiguous and non-contiguous.

- The main memory is a combination of two main portions- **one for the operating system and other for the user program.**
- We can implement/achieve contiguous memory allocation by d**ividing the memory partitions into fixed size partitions.**

Advantages of Contiguous Memory Allocation

- **Faster Execution:** Less time is consumed in the execution of processes due to the fact that **memory is provided in a sequential manner.**
- **Minimal Overhead:** There are fewer address translations, which all contribute to the efficiency of the system.
- **Easier to Control:** Organization of the memory in the operating system is sequential, which makes it possible to have efficient control of the processes.

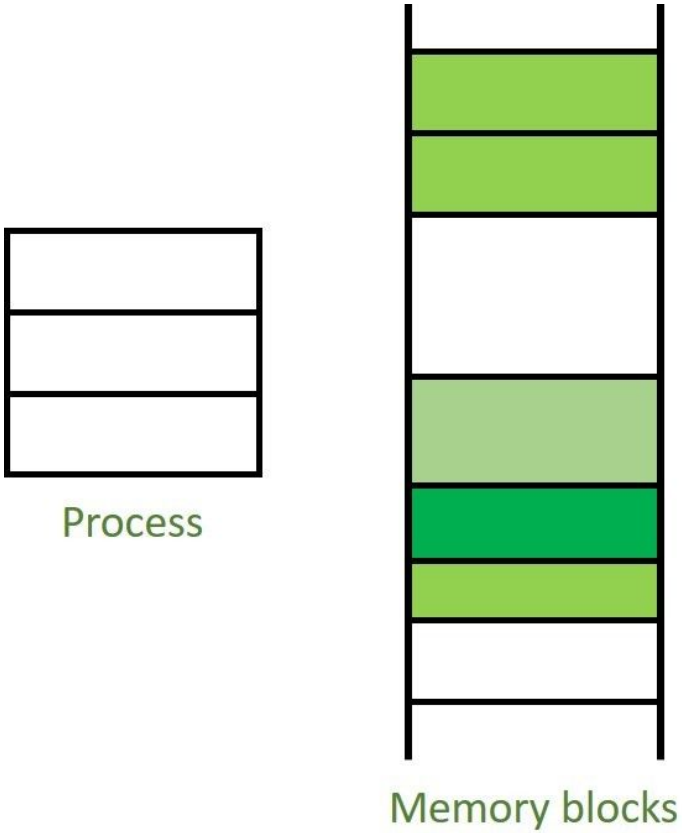
Introduction: Contiguous

Disadvantages of Contiguous Memory Allocation

- **Internal and External Fragmentation: I**t was noted that one way that memory blocks can be implemented is that they may become fragmented, which results in inefficiency.
- **Limited Multiprogramming:** The disadvantage of fixed partitioning is that it hinders the ability to control and allocate several processes.
- **Wastage of Memory:** With fixed partitions, memory can be wasted, especially where its usage is not optimal as required.

Introduction: Contiguous

- **What is Non-Contiguous Memory Allocation?**
- Non-Contiguous memory allocation is basically a method on the contrary to contiguous allocation method, **allocates the memory space present in different locations to the process as per it's requirements.**
- As all the available memory space is in a distributed pattern so the freely available memory space is also scattered here and there. This technique of memory allocation **helps to reduce the wastage of memory, which eventually gives rise to** Internal and external fragmentation.



Noncontiguous Memory Allocation

Introduction: Non-Contiguous

Advantages of Non-Contiguous Memory Allocation

- **No Memory Wastage:** Dynamic allocation makes sure that memory is used as is available and that there is no memory wasted.
- **Increased Multiprogramming:** Non-contiguous allocation is advantageous where higher degrees of multiprogramming are feasible.
- **Reduced fragmentation:** Based on its primary uses, it mainly minimizes internal fragmentation.

Disadvantages of Non-Contiguous Memory Allocation

- **Slower Execution:** Tasks could be longer in duration because memory is frequently split into blocks.

Difference Between Contiguous and Non-contiguous Memory Allocation

Contiguous Memory Allocation	Non-Contiguous Memory Allocation
Contiguous memory allocation **allocates consecutive blocks** of memory to a file/process.	Non-Contiguous memory allocation **allocates separate blocks of memory** to a file/process.
Faster in Execution.	**Slower** in Execution.
It is **easier for the OS** to control.	It is **difficult for the OS** to control.
Overhead is **minimum as not much address translations** are there while executing a process.	**More Overheads are there as there** are more address translations.
Both **<u>Internal fragmentation and external fragmentation</u> o**ccurs in Contiguous memory allocation method.	Only **External fragmentation occurs in** Non-Contiguous memory allocation method.

It includes **single partition allocation and multi-partition allocation.**	It includes **paging and segmentation**.
Wastage of memory is there.	**No memory wastage** is there.
In contiguous memory allocation, s**wapped-in processes** are arranged in the originally allocated space.	In non-contiguous memory allocation**, swapped-in processes can be arranged in any place in the memory.**
It is of two types: 1. Fixed(or static) partitioning 2. Dynamic partitioning	**It is of five types:** 1. Paging 2. Multilevel Paging 3. Inverted Paging 4. Segmentation 5. Segmented Paging
It could be **visualized and implemented using Arrays.**	It could be **implemented using Linked Lists.**
Degree of **multiprogramming is fixed as fixed partitions**	Degree of **multiprogramming is not fixed**

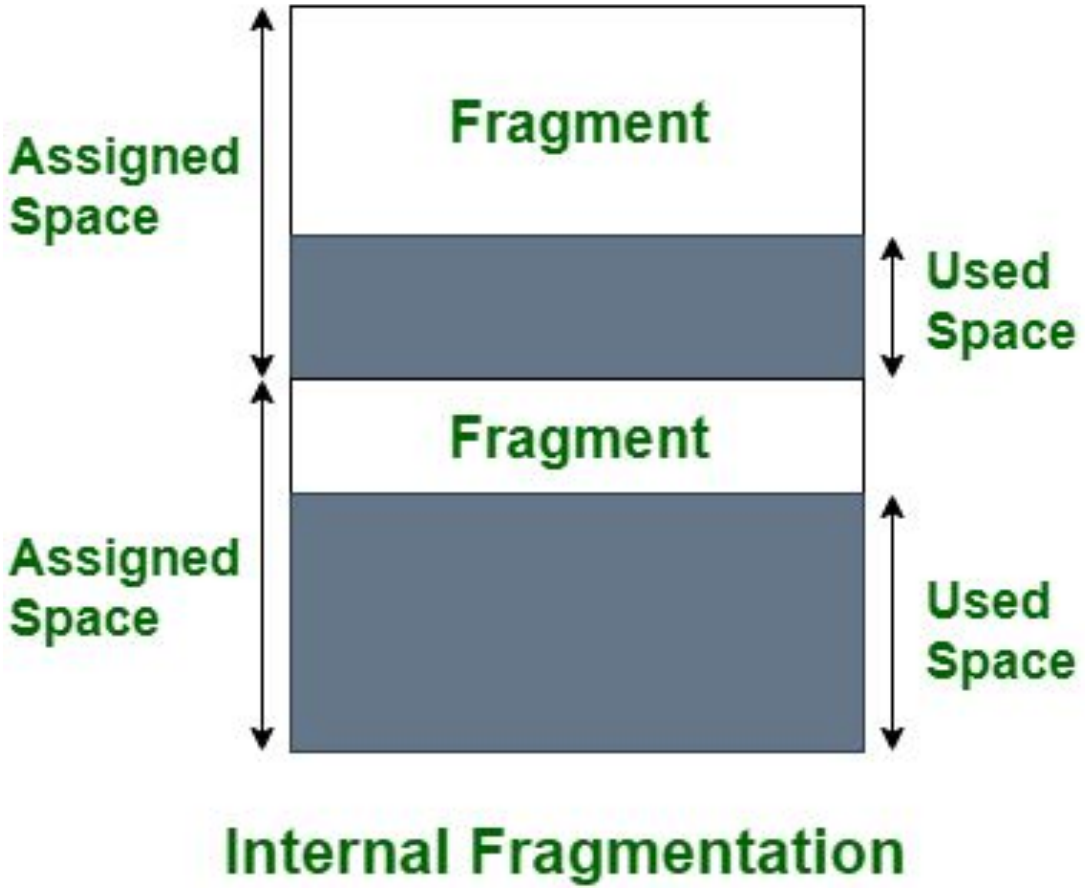
Fragmentation: Internal and External fragmentation.

What is Internal Fragmentation?

- Internal fragmentation happens when the memory is split into **mounted-sized blocks. Whenever a method is requested for the memory, the mounted-sized block is allotted to the method.**
- In the case where the memory allocated to the method is somewhat larger than the memory requested, then the **difference between allotted and requested memory is called internal fragmentation.**
- We **fixed the sizes of the memory blocks, which has caused this issue.** If we use **dynamic partitioning to allot space to the process, this issue can be solved.**

Fragmentation: Internal and External fragmentation.

What is Internal Fragmentation?



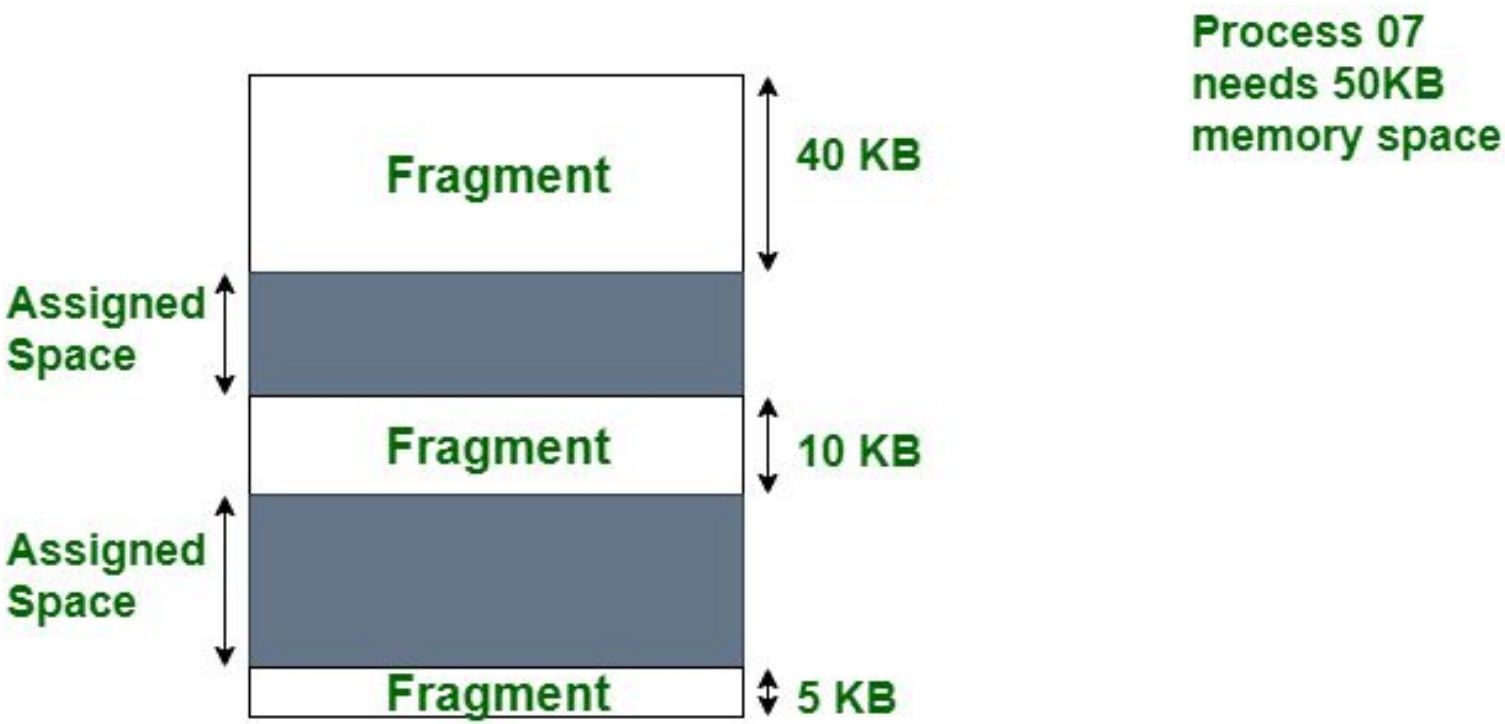
Fragmentation: Internal and External fragmentation.

What is External Fragmentation?

- External fragmentation happens when there's a **sufficient quantity of area within the memory to satisfy the memory request of a method.**
- However, the process's memory **request cannot be fulfilled because the memory offered is in a non-contiguous manner.**
- Whether you apply a **first-fit or best-fit memory allocation strategy it'll cause external fragmentation.**

Fragmentation: Internal and External fragmentation.

What is External Fragmentation?

- 

Difference Between Internal Fragmentation and External Fragmentation

Internal fragmentation	External fragmentation
In internal fragmentation **fixed-sized memory,** blocks square measure appointed to process.	In external fragmentation, **variable-sized memory blocks** square measure appointed to the method.
Internal fragmentation happens **when the method or process is smaller than the memory.**	External fragmentation **happens when the method or process is removed.**
The solution of internal fragmentation is **the <u>best-fit block</u>.**	The solution to external fragmentation is **compaction and <u>paging</u>.**
Internal fragmentation occurs when **memory is divided into <u>fixed-sized partitions.</u>**	External fragmentation occurs when memory is **divided into variable size partitions based on the size of processes.**

Difference Between Internal Fragmentation and External Fragmentation

Internal fragmentation	External fragmentation
The difference **between memory allocated and required space or memory is called Internal fragmentation.**	The unused spaces formed **between <u>non-contiguous memory</u> frag**ments are too small to serve a new process, which is called External fragmentation.
Internal fragmentation occurs with paging and fixed partitioning.	External fragmentation occurs with segmentation and <u>dynamic partitioning</u>.
It occurs on the allocation of a process to a partition greater than the process's requirement. The leftover space causes degradation system performance.	It occurs on the allocation of a process to a partition greater which is exactly the same memory space as it is required.
It occurs in worst fit <u>memory allocation method</u>.	It occurs in best fit and first fit memory allocation method.

Memory allocation strategies: First Fit, Best Fit, Worst Fit

- In an operating system (OS), memory management is a critical function that ensures efficient allocation and utilization of memory resources.
- When processes are initiated, they require execution time and storage space, which is allocated in memory blocks.
- The OS uses various memory allocation algorithms to manage the assignment of memory to these incoming processes.

The following are the most used algorithms:

- **1. First Fit**
- **2. Best Fit**
- **3. Worst Fit**
- **4. Next Fit**

First-Fit Memory Allocation

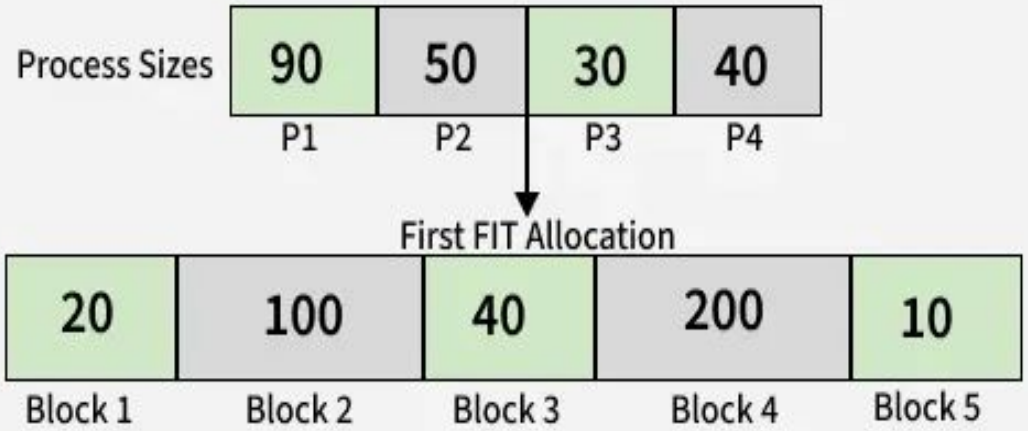
The **First Fit** method works by **searching through memory blocks from the beginning, looking for the first block that can fit the process.**

- For any process Pn, the operating system **looks through the available memory blocks.**
- It allocates the **first block that is free and large enough** to hold the process.

In simple terms, First Fit simply finds the **first available block that can fit the process and assigns it there. I**t's a straightforward and fast way to allocate memory.

First-Fit Memory Allocation

First Fit Allocation in OS



	Size	Allocated	Memory Wastage After Process Occupies
Process 1	90	Block 2	100-90 = 10
Process 2	50	Block 4	200-50 = 150
Process 3	30	Block 3	40-30-10
Process 4	40	Unallocated	-

P4 remains Unallocated

Algorithm for First-Fit Memory Allocation

Steps:

1. Start with the first process in the list.

2. For each process Pn, do the following:

- **Step 1**: Search through the available memory blocks, starting from the first block.
- **Step 2**: Check if the current memory block is free and has enough space to accommodate Pn.
- **Step 3**: If a suitable block is found, allocate it to the process and update the block's status (mark it as allocated).
- **Step 4**: If no suitable block is found, mark the process as unallocated or waiting.

3. Repeat the above steps for each process.

Advantages of First Fit Algorithm

The First Fit algorithm in operating systems offers several benefits:

- It is **straightforward to implement and easy to understand**, making it ideal for systems with limited computational power.
- Memory **can be allocated quickly** when a suitable free block is found at the start.
- When **processes have similar memory sizes, First Fit can help minimize fragmentation** by utilizing the first available block that fits the process.

Disadvantages of First Fit Algorithm

- Over time, First Fit can lead to both external fragmentation, where small free memory blocks are scattered, and internal fragmentation, where **allocated memory exceeds the process's requirement, wasting space.**
- It may not always allocate memory in the most efficient manner, leading to suboptimal use of available memory.
- For large processes, **First Fit can be less efficient,** as it may need to search through numerous smaller blocks before finding an appropriate one, which can slow down memory allocation.

Best-Fit Memory Allocation

- Best-Fit Allocation is a memory allocation technique used in operating systems **to allocate memory to a process.**
- In Best-Fit, the operating system **searches through the list of free blocks of memory to find the block that is closest in size to the memory request** from the process.
- Once a **suitable block is found,** the operating system splits the block into **two parts: the portion that will be allocated to the process, and the remaining free block.**

Best-Fit Memory Allocation

Job Number	Memory Requested
J1	20 K
J2	200 K
J3	500 K
J4	50 K

Memory location	Memory block size	Job number	Job size	Status	Internal fragmentation
10567	30 K	J1	20 K	Busy	10 K
30457	50 K	J4	50 K	Busy	None
300875	200 K	J2	200 K	Busy	None
809567	700 K	J3	500 K	Busy	200 K
Total available :	980 K	Total used :	770 K		210 K

Best-Fit Memory Allocation Algorithm

a. The operating system maintains a list of all free memory blocks available in the system.
b. When a process requests memory, the operating system searches the list for the smallest free block of memory that is large enough to accommodate the process.
c. If a suitable block is found, the process is allocated memory from that block.
d. If no suitable block is found, the operating system can either wait until a suitable block becomes available or request additional memory from the system.
e. The best-fit allocation algorithm has the advantage of minimizing external fragmentation, as it searches for the smallest free block of memory that can accommodate a process. However, it can also lead to more internal fragmentation, as processes may not use the entire memory block allocated to them.

Advantages Best-Fit Allocation

- Best-Fit Allocation include **improved memory utilization, as it allocates the smallest block of memory** that is sufficient to accommodate the memory request from the process.
- Additionally, B**est-Fit can also help to reduce memory fragmentation,** as it tends to allocate smaller blocks of memory that are less likely to become fragmented.

Disadvantages Best-Fit Allocation

- Best-Fit Allocation include **increased computational overhead, as the search for the best-fit block of memory can be time-consuming** and requires a more complex search algorithm.
- Additionally, **Best-Fit may also result in increased fragmentation,** as it may leave smaller blocks of memory scattered throughout the memory space.

Worst-Fit Memory Allocation

- In Worst-Fit Memory Allocation allocation technique, **the process traverses the whole memory and always search for the largest hole/partition**, and then the process is placed in that hole/partition.
- It is a **slow process because it has to traverse the entire memory to search the largest hole.**

Algorithm for Worst Fit

- Step 1: Input the list of memory blocks along with their sizes.
- Step 2: Input the list of processes with their sizes.
- Step 3: For each process, check for the largest memory block (i.e., the block with the maximum size) that is large enough to accommodate the process.
- Step 4: If such a block is found, allocate the process to that block and update the block size.
- Step 5: If no suitable block is found, leave the process unallocated and move on to the next process.
- Step 6: Repeat the above steps for all processes.
- Step 7: Stop.

Example for Worst Fit

Advantages of Worst-Fit Allocation

- **Reduces Chances of Small Fragments:** By allocating the largest available block, worst fit leaves larger leftover fragments, which are more likely to be useful for future allocations.
- **Simple to Implement:** The logic is straightforward: find the largest block that can fit the process and allocate it.
- **Efficient for Large Processes:** It always looks for the biggest space, large processes may find better fits compared to other strategies like Best Fit or First Fit.

Disadvantages of Worst-Fit Allocation

- **External Fragmentation:** Worst-Fit Allocation can lead to significant external fragmentation. By allocating the largest available block, it often leaves behind many smaller, unusable fragments of memory.
- **Inefficient Memory Usage:** Allocating the largest block to a process, even if it's much larger than required, can lead to inefficient memory usage.
- **Lower Allocation Time:** Worst-Fit requires the operating system to search for the largest available block, which can be slower than methods like First-Fit or Next-Fit, especially if there are many free blocks.

Next Fit Memory Allocation

What is Next Fit?

- The next fit is a **modified version of 'first fit'**. It begins as the first fit to find a free partition but **when called next time it starts searching from where it left off, not from the beginning.**
- This policy makes use of a roving pointer. The pointer moves along the memory chain to search for the next fit. This helps in, **to avoid the usage of memory always from the head (beginning) of the free blockchain.**
-

Next Fit Memory Allocation

-

Example:

Input :

```
blockSize[] = {5, 10, 20};
processSize[] = {10, 20, 30};
```

Output:

Process No.	Process Size	Block no.
1	10	2
2	20	3
3	30	Not Allocated

Next Fit Memory Allocation

Algorithm:

1. Input the number of **memory blocks and their sizes and initializes all the blocks as free.**
2. Input the **number of processes and their sizes.**
3. Start by picking each process and check if it can be assigned to the current block, if yes, allocate the required memory and check for next process **but from the block where we left not from starting.**
4. If the **current block size is smaller then keep checking the further blocks.**

Silberschatz, Galvin and Gagne ©2018

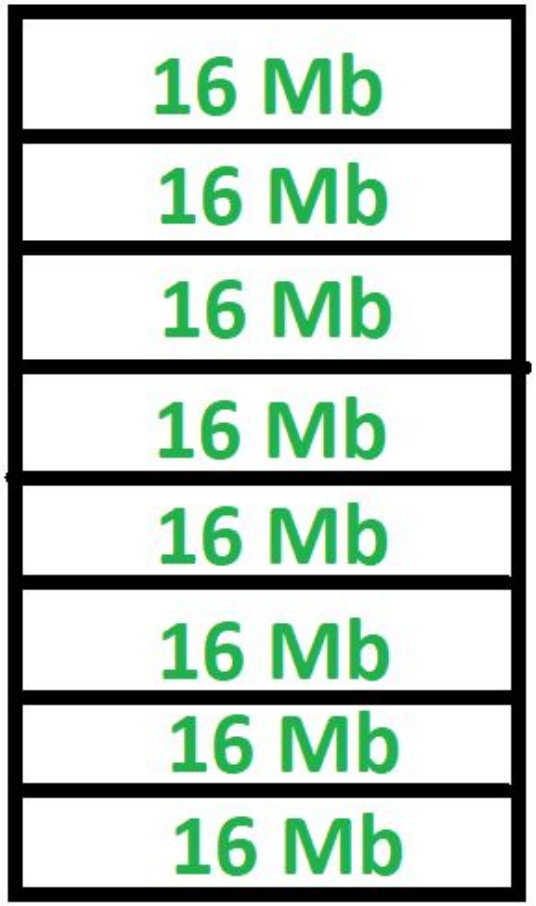
Partitioning: Fixed and Dynamic partitions.

What is Fixed Partitioning?

- Fixed Partitioning is a **contiguous memory management technique in which the main memory is divided into fixed sized partitions which can be of equal or unequal size.**
- Whenever we have t**o allocate a process memory then a free partition that is big enough to hold the process is found.** Then the memory is allocated to the process.
- If there is **no free space available then the process waits in the queue to be allocated memory.**
- It is one of the most oldest memory management technique which is easy to implement.

Partitioning: Fixed and Dynamic partitions.

What is Fixed Partitioning?



Partitioning: Fixed and Dynamic partitions.

Advantages of Fixed Partitioning

Simplicity: Since partitions are already defined, it is easy to assign memory to a process especially when it is designed that way.

This makes partitioning method very easy to implement especially since it only requires partitioning of a few segments.

Disadvantages of Fixed Partitioning

Internal Fragmentation: If a process is not able to fill a partition it uses then there is space left in that partition which is not utilized.

Inefficient Memory Utilization: The size of partitions is fixed and therefore large processes might occupy more space while small processes occupy inadequate large space.

Limited Process Size: Quantities greater than the size of the largest partition cannot be performed; this decreases flexibility.

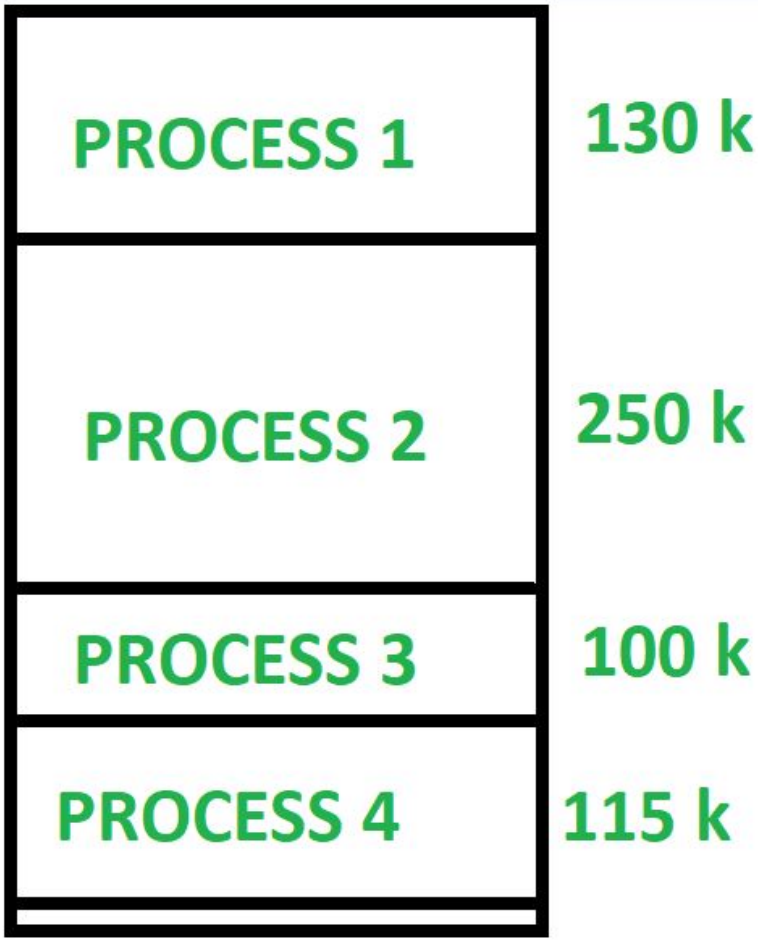
Partitioning: Fixed and Dynamic partitions.

What is Variable/Dynamic Partitioning?

- Variable Partitioning is a **contiguous memory management technique in which the main memory is not divided into partitions and the process is allocated a chunk of free memory that is big enough for it to fit.**
- The space ***which is left is considered as the free space which can be further used by other processes***. It also provides the concept of compaction.
- In **compaction the spaces that are free and the spaces which not allocated to the process are combined and single large memory space is made.**

Partitioning: Fixed and Dynamic partitions.

What is Variable/Dynamic Partitioning?



Partitioning: Fixed and Dynamic partitions.

Advantages of Variable Partitioning

Efficient Memory Utilization: Memory is allocated dynamically; this means that it can be used sparingly thus avoiding the use of much space on the available memory.

No Process Size Limitation: One advantage of this is an absence of sharp partitions because memory is allocated as per the processes requirements.

Higher Degree of Multi-programming: It also stated that process flexibility for memory can be suitable for more process.

Disadvantages of Variable Partitioning

External Fragmentation: Any time memory space is granted and then recalled for someone else to free, space of maximum un-utilized portions of memory are created termed as external fragmentation leading to poor memory utilization.

Complexity: Fixed partitioning and its related concepts are far simpler to handle and control as compared to variable partitions and external fragmentation.

Need for Compaction: To avoid this fragmentation, compaction is needed, in which the free areas are grouped together to form bigger spaces in the memory, but this takes a lot of time and consumes many resources.

Partitioning: Fixed and Dynamic partitions.

Fixed Partitioning	Variable Partitioning
Memory is divided into fixed-sized partitions.	Memory is allocated dynamically in varying sizes.
Only one process can be placed in each partition.	A process is allocated a chunk of free memory as needed.
Inefficient memory utilization due to internal fragmentation.	More efficient memory utilization with less internal fragmentation.
Internal and external fragmentation occur.	Only external fragmentation occurs.
Limited degree of multi-programming.	Higher degree of multi-programming due to flexible allocation.
Easier to implement.	More complex to implement.
Restricts process size based on partition size.	No size limitation on processes.

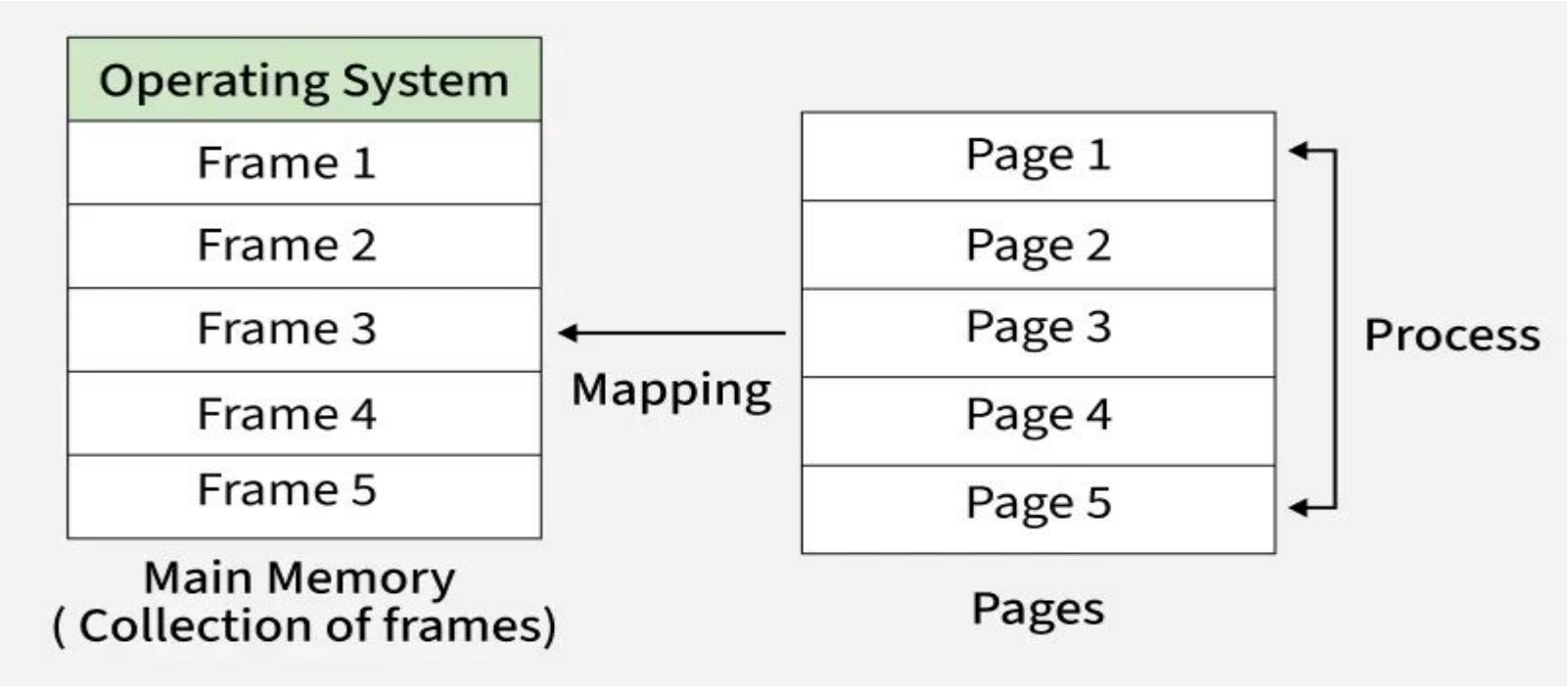
Paging in Operating System

Paging is the process of moving parts of a program, called pages, from **secondary storage into the main memory (RAM).** The main idea behind paging is **to break a program into smaller fixed-size blocks called pages.**

It has following advantages.

- Process **does not have to be allocated in a contiguous** memory space.
- The whole process **does not have to be in main memory, some pages can be present and some pages can be loaded when needed.**
- It allows more processes and even processes larger than main memory to run.
- Memory allocation is simplified as memory is always allocated in fixed sized pages.

Paging in Operating System



To keep track of where each page is stored in memory, the operating system uses a page table. This table shows the c**onnection between the logical page numbers and the physical page frames (actual locations in RAM).**

Paging in Memory Management

Paging addresses common challenges in allocating and managing memory efficiently.
Why paging is needed as a Memory Management technique:

- Memory isn't always available in a single block: Programs **often need more memory than what is available in a single continuous block.**
- **Paging breaks memory into smaller, fixed-size pieces, making it easier to allocate scattered free spaces.**
- Processes size can increase or decrease: **programs don't need to occupy continuous memory**, so they can **grow dynamically** without the need to be moved.

Paging in Memory Management

Terminologies Associated with Memory Control

Logical Address Space or Virtual Address Space:
The Logical Address Space, refers to the set of all possible logical addresses that a process can generate during its execution.

Physical Address Space:
The Physical Address Space refers to the total range of addresses available in a computer's physical memory (RAM).

Important Features of Paging

Logical to physical address mapping: Paging divides a process's logical address space into fixed-size pages. Each page maps to a frame in physical memory, enabling flexible memory management.

Fixed page and frame size: Pages and frames have the same fixed size. This simplifies memory management and improves system performance.

Page table entries: Each logical page is represented by a page table entry (PTE). A PTE stores the corresponding frame number and control bits.

Number of page table entries: The page table has one entry per logical page. Thus, its size equals the number of pages in the process's address space.

Page table stored in main memory: The page table is kept in main memory. This can add overhead when processes are swapped in or out.

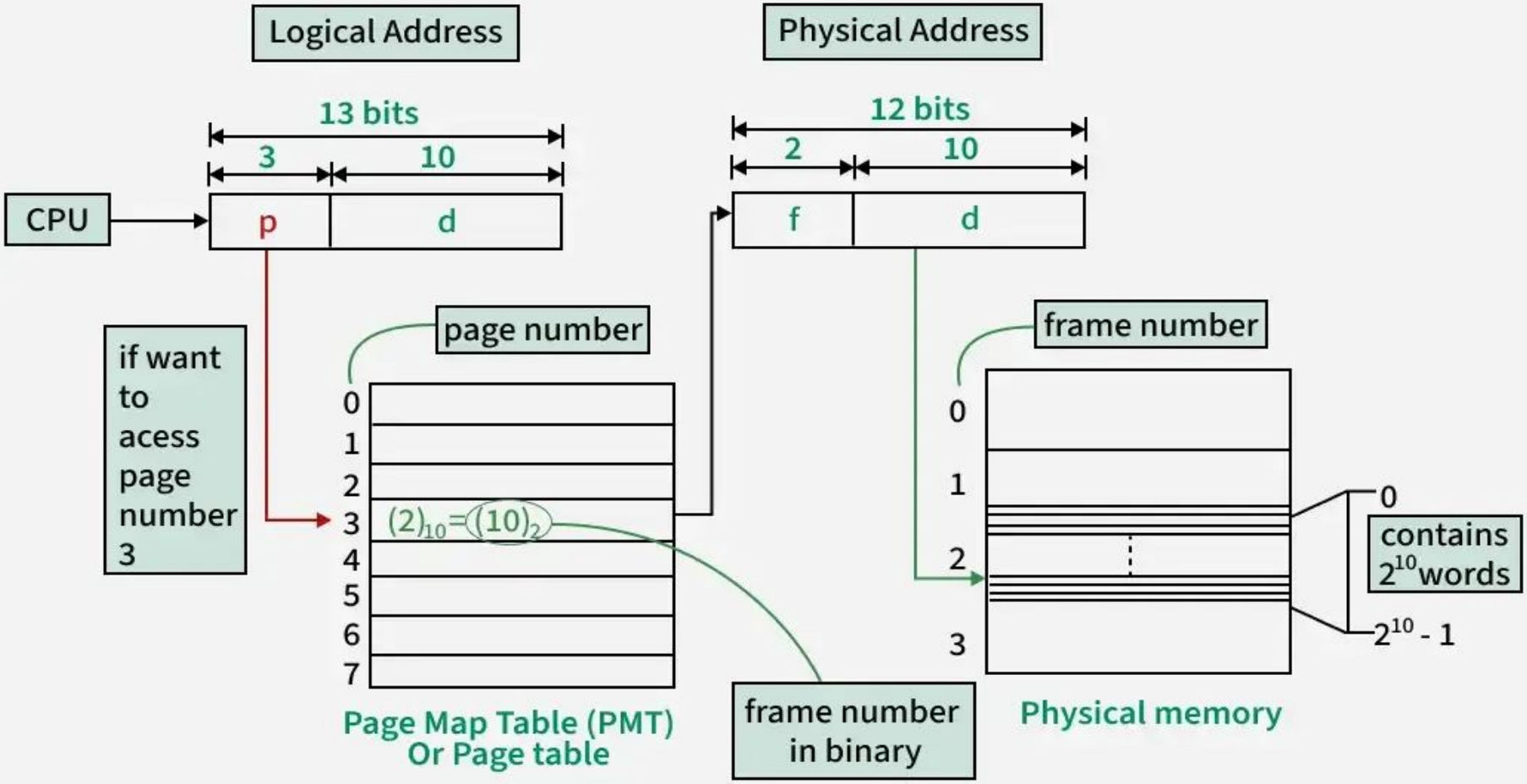
Working of Paging

When a process requests memory, the operating system allocates one or more page frames to the process and **maps the process's logical pages to the physical page frames.** When a program runs, its p**ages are loaded into any available frames** in the physical memory.

Steps Involved in Paging :

Step 1 Divide Memory : Logical -> Pages, Physical -> Frames .
Step 2 Allocate Pages : Load pages into available frames.
Step 3 Page Table : Map logical pages to physical frames.
Step 4 Translate Address : Convert logical to physical address.
Step 5 Handle Page Fault : Load missing pages from disk.
Step 6 Run Program : CPU uses page table during execution.

Working of Paging



Working of Paging

- The Physical Address Space is conceptually divided into a number of fixed-size blocks, called frames.
- The Logical Address Space is also split into fixed-size blocks, called pages.
- Page Size = Frame Size

Working of Paging Example

- **Physical Address =** 12 bits, then Physical Address Space = 4 K words
- **Logical Address =** 13 bits, then Logical Address Space = 8 K words
- **Page size = frame size = 1 K words (assumption)**
- **Number of frames =** Physical Address Space / Frame Size = 4K / 1K = 4 = 22
- **Number of Pages =** Logical Address Space / Page Size = 8K / 1K = 23

Working of Paging Example

The address generated by the CPU is divided into:

- **Page number(p):** Number of bits required to represent the pages in Logical Address Space or Page number
- **Page offset(d):** Number of bits required to represent a particular word in a page or page size of Logical Address Space or word number of a page or page offset.
- A Physical Address is divided into two main parts:
- **Frame Number(f):** Number of bits required to represent the frame of Physical Address Space or Frame number frame
- **Frame Offset(d): N**umber of bits required to represent a particular word in a frame or frame size of Physical Address Space or word number of a frame or frame offset.

Physical Address = (Frame Number << Number of Bits in Frame Offset) + Frame Offset where "<<" represents a bitwise left shift operation.

Segmentation in Operating System

- Segmentation is a memory management technique w**here a process is divided into variable-sized chunks called segments.**
- Unlike paging, **segmentation matches the user's logical view of a program (functions, arrays, modules) to physical memory.**
- It reflects the u**ser's view of memory rather than the computer's physical organization, making it easier to manage** and protect processes.

Key Features of Segmentation

1. Variable-sized divisions: Segments can have different lengths, depending on the program's requirements.

2. Logical division of memory: Segments represent meaningful units like **code, stack, data, or modules.**

3. Two-part address: A logical address consists of:

- **Segment number (s):** Identifies which segment is being referred to.
- **Offset (d):** Specifies the exact location within that segment.
- **Logical Address =** ⟨Segment number, Offset⟩

4. Protection and sharing: Different segments can have access rights (read, write, execute) and can be shared among processes.

5. No internal fragmentation: Since segments are not fixed in size.

6. External fragmentation: Can occur when free memory is divided into small scattered blocks.

Types of Segmentation in Operating Systems

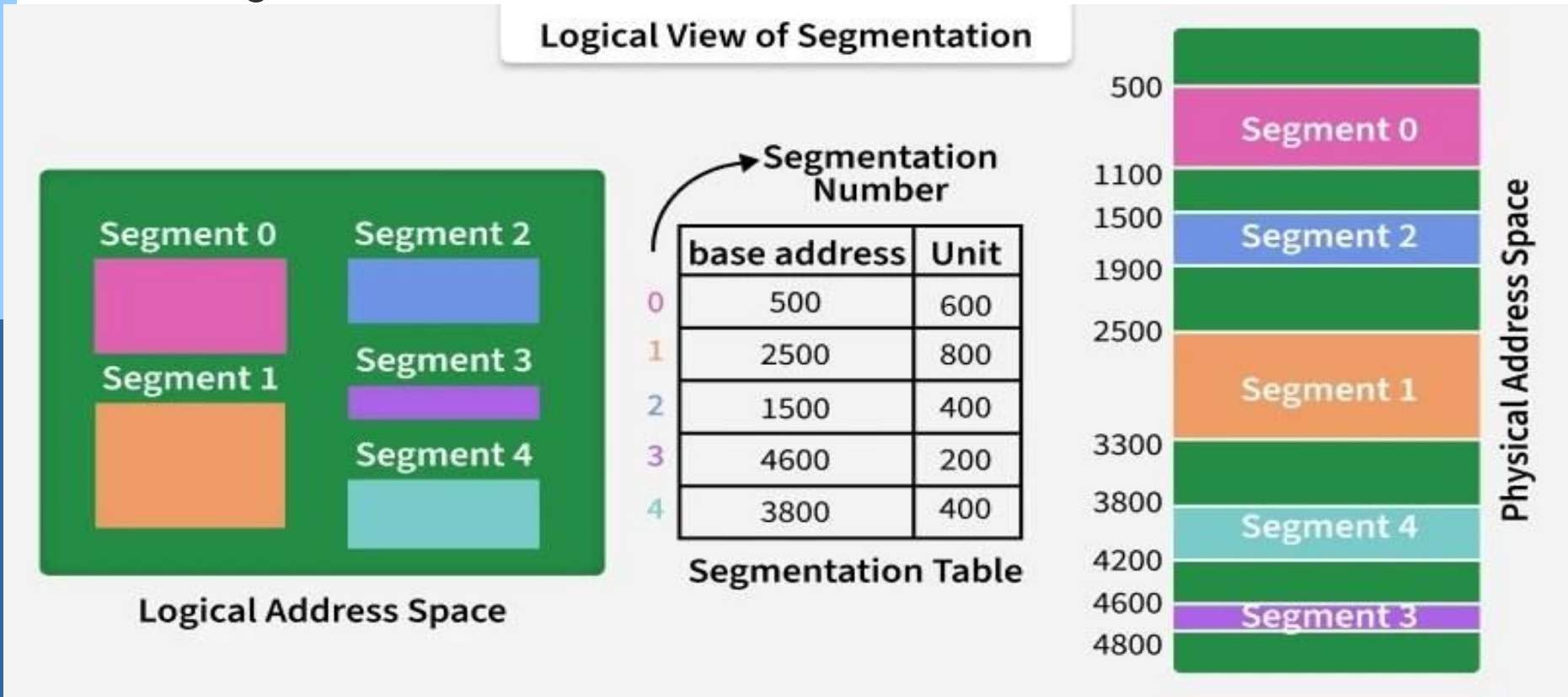
Virtual Memory Segmentation: Each process is divided into a number of segments, but the segmentation is not done all at once. This segmentation may or may not take place at the run time of the program.

Simple Segmentation: Each process is divided into a number of segments, all of which are loaded into memory at run time, though not necessarily contiguously.

What is Segment Table?

It maps a two-dimensional Logical address into a one-dimensional Physical address. It's each table entry has:

- Base Address: It contains the starting physical address where the segments reside in memory.
- Segment Limit: Also known as segment offset. It specifies the length of the segment.



What is Segment Table?

It maps a two-dimensional Logical address into a one-dimensional Physical address. It's each table entry has:

- Base Address: It contains the starting physical address where the segments reside in memory.
- Segment Limit: Also known as segment offset. It specifies the length of the segment.

Virtual Memory

- **Virtual memory** is a mechanism used to manage memory using hardware and software.
- It is a part of the <u>secondary storage</u> that gives the user the illusion that it is a part of the main memory.
- It helps in running multiple applications with low main memory and increases the degree of multiprogramming in systems.
- It is commonly implemented using <u>demand paging</u>.

Virtual Memory

- Virtual memory is a part of the system's secondary memory that acts and gives us a feel as if it is a part of the main memory.

- Virtual memory **allows a system** to execute heavier applications or multiple applications simultaneously without exhausting the RAM (Random Access Memory).

History of Virtual Memory

- The first virtual memory was developed in **1959**. It was disapproved.

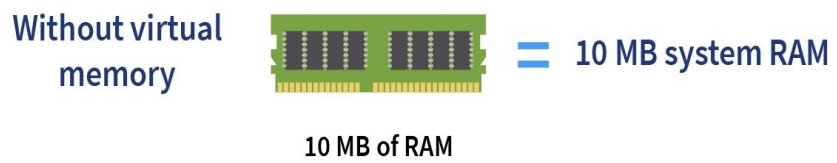
- By the late **1970s,** the idea of virtual memory was properly developed and acquire into commercial computers.

- However, it was not until 1985, that <u>Intel introduced virtual memory</u>, which marked the beginning of virtual memory in **personal computers**. That's how virtual memory has its place in our daily lives.

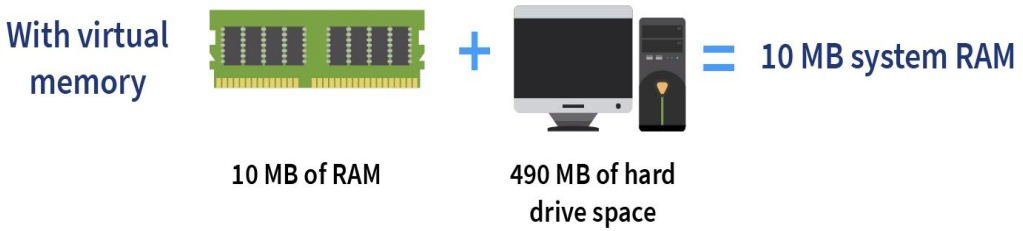
How Virtual Memory Works?

- Assume that an operating system uses **500 MB of RAM** to hold all of the running processes. However, there is now only 10 MB of actual capacity accessible on the RAM.

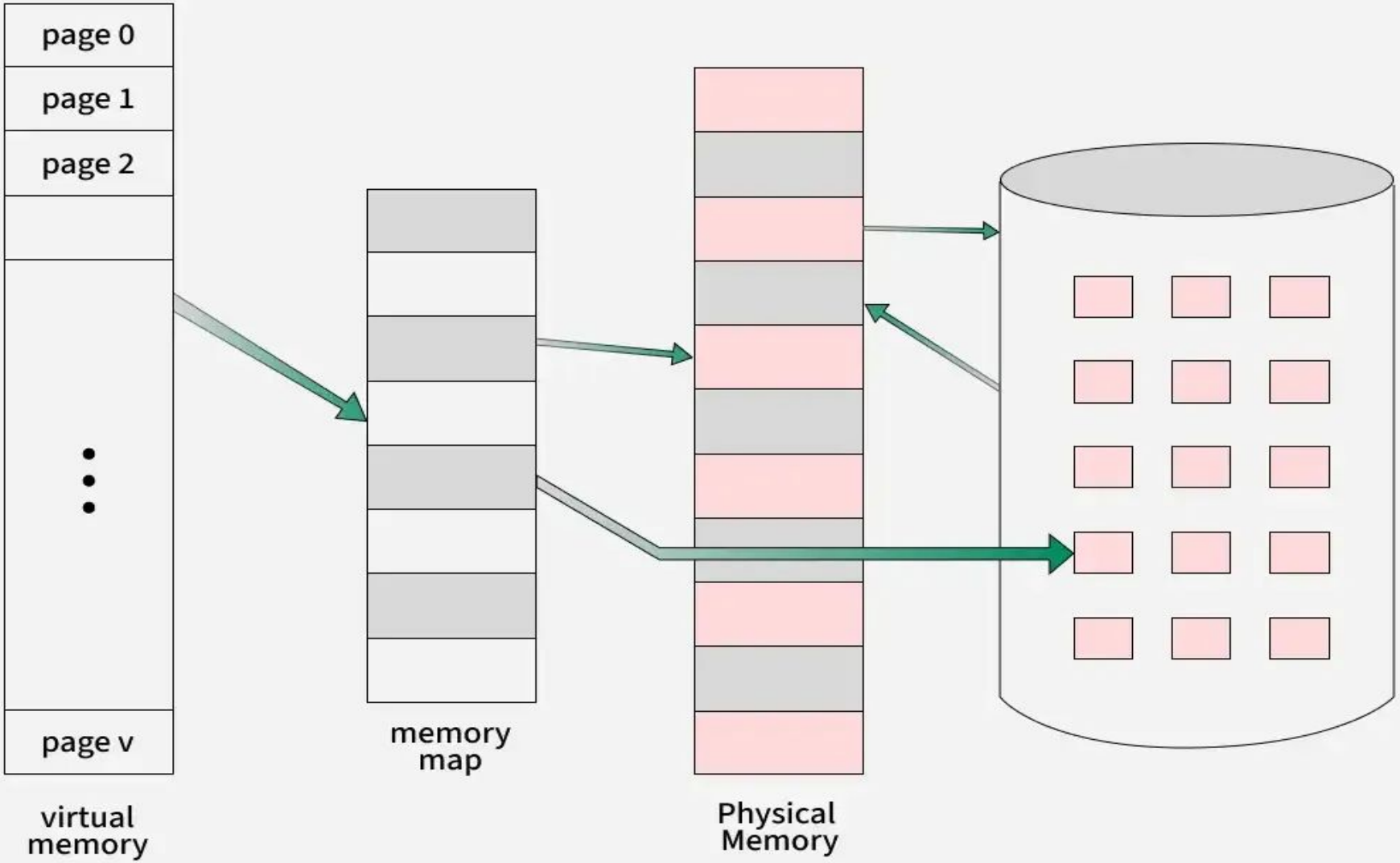
- The **operating system** will then allocate 490 MB of virtual memory and manage it with an application called the Virtual Memory Manager (VMM).



- As a result, the **VMM** will generate a 490 MB file on the hard disc to contain the extra RAM that is necessary.



How Virtual Memory Works?



How Virtual Memory Works?

- When running multiple heavy applications at once, the system's RAM may get overloaded.

- **To mitigate this issue**, some data stored in RAM that isn't being actively used *can be temporarily relocated to virtual memory*.

- This frees up RAM space, which may then be utilized to store data that the system will need to access.

Demand Paging

- **Demand paging** is a technique used in virtual memory systems where pages enter main memory only when requested or needed by the CPU.

- OS loads only the necessary pages of a program into memory at runtime, instead of loading the entire program into memory at the start.

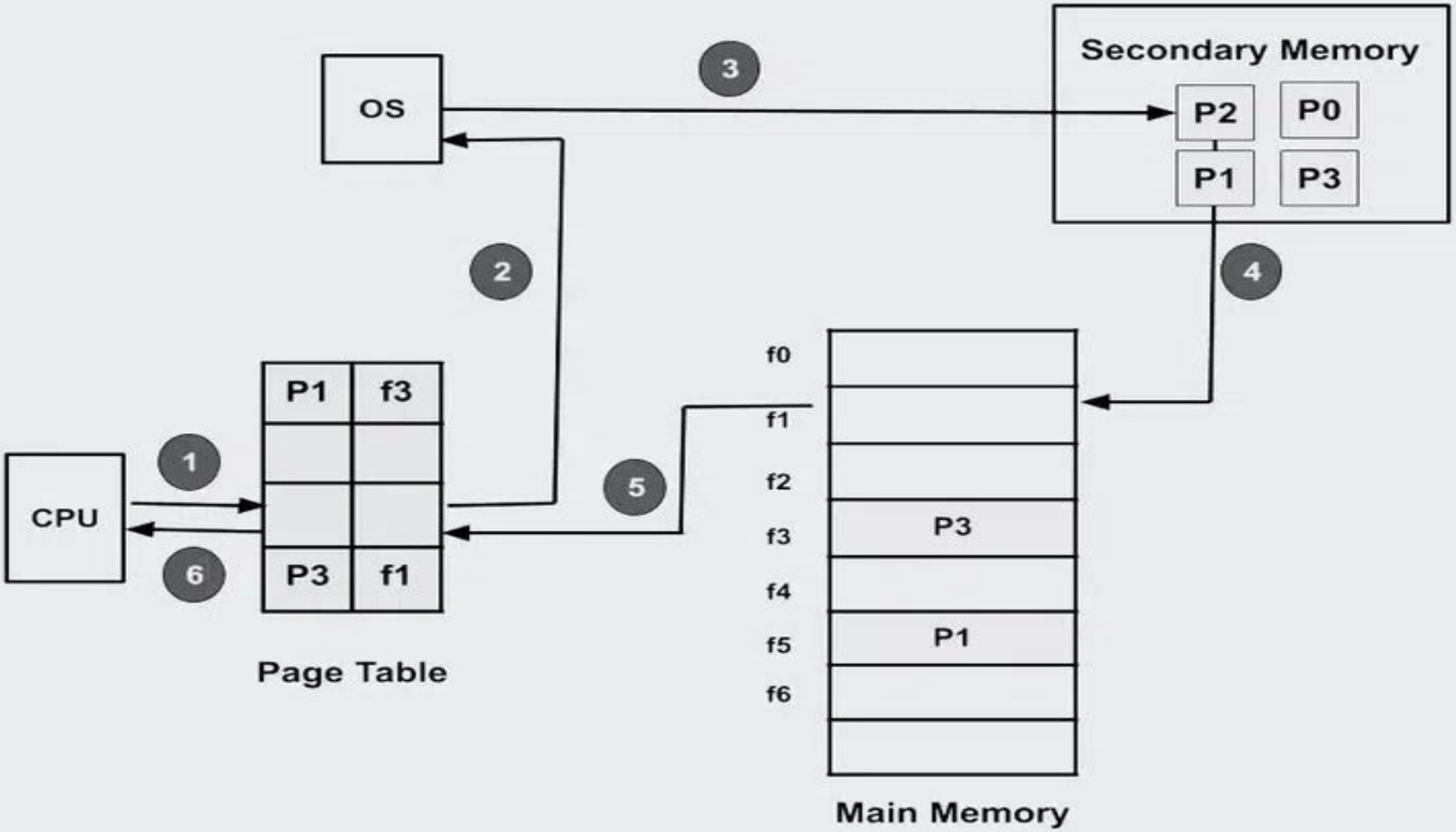
- **A page fault occurred** when the program needed to access a page that is not currently in memory.

Demand Paging

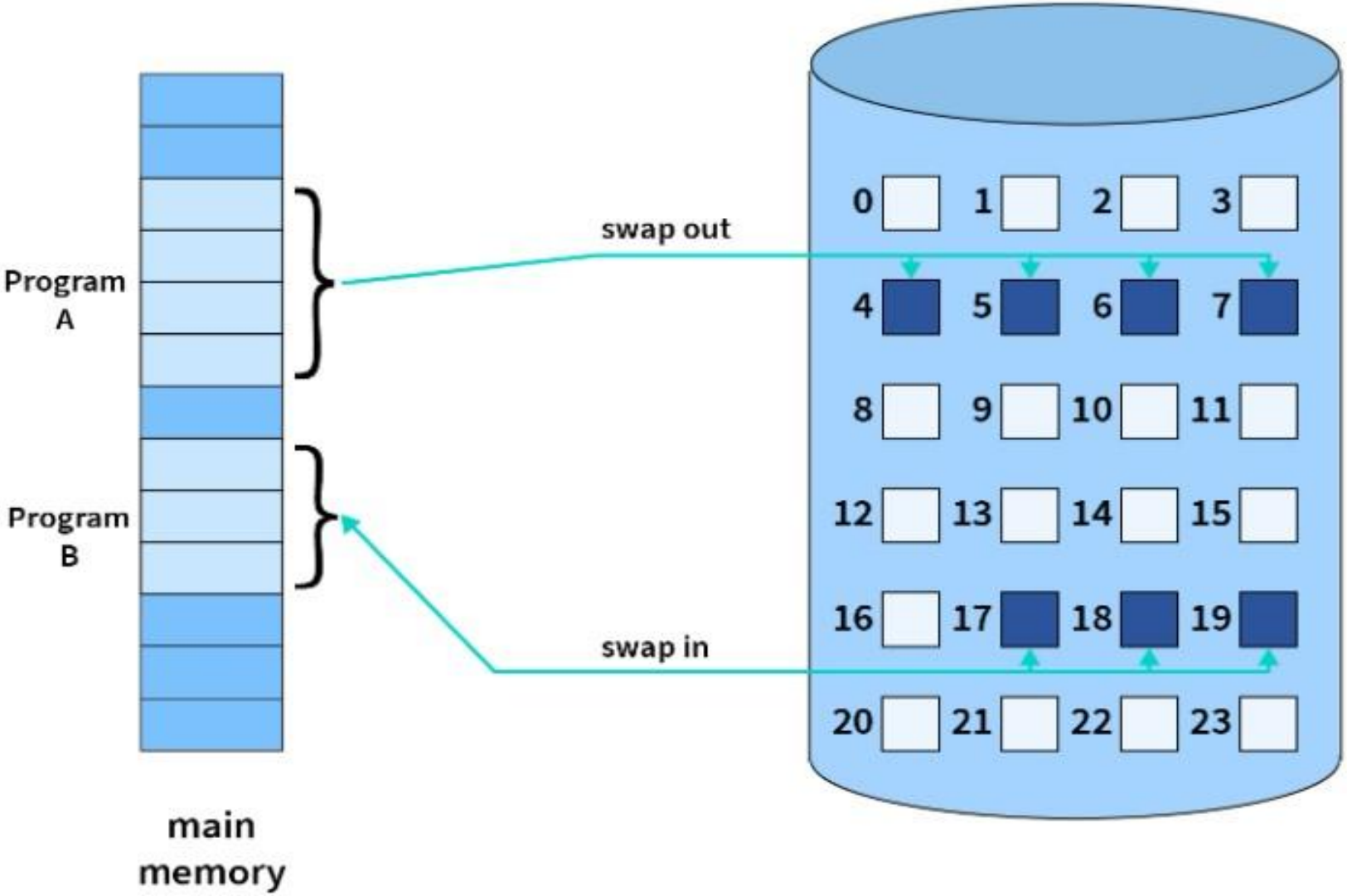
- **The operating system** then loads the required pages from the disk into memory and updates the page tables accordingly.

- This process is transparent to the running program and it continues to run as if the page had always been in memory.

Working of Demand Paging



Working of Demand Paging



Working of Demand Paging

- **Program Execution:** Upon launching a program, the operating system allocates a certain amount of memory to the program and establishes a process for it.

- **Creating Page Tables:** To keep track of which program pages are currently in memory and which are on disk, the operating system makes page tables for each process.

- **Handling Page Fault:** When a program tries to access a page that isn't in memory at the moment, a page fault happens. In order to determine whether the necessary page is on disk, the operating system pauses the application and consults the page tables.

Working of Demand Paging

- **Page Fetch:**

The operating system loads the necessary page into memory by retrieving it from the disk if it is there.

The page's new location in memory is then reflected in the page table.

- **Resuming The Program:**

The operating system picks up where it left off when the necessary pages are loaded into memory.

Working of Demand Paging

- **Page Replacement:**

If there is not enough free memory to hold all the pages a program needs, the operating system may need to replace one or more pages currently in memory with pages currently in memory.

The page replacement algorithm used by the operating system determines which pages are selected for replacement.

- **Page Cleanup:** When a process terminates, the operating system frees the memory allocated to the process and cleans up the corresponding entries in the page tables.

Advantages of Virtual Memory

- Supports Multiprogramming & Larger Programs
- Maximizes Application Capacity
- Eliminates Physical Memory Limitations
- Improves CPU & System Performance
- Enhances Memory Management Efficiency

Disadvantages of Virtual Memory

- Slower Performance
- Risk of Data Loss
- More Complex System

Page Replacement Algorithm

- In an operating system that uses paging, **a page replacement algorithm is needed** when a page fault occurs and no free page frame is available.

- In this case, one of the existing pages in memory must be replaced with the new page.

1. First In First Out (FIFO)
2. Optimal Page replacement
3. Least Recently Used (LRU)

Page Replacement Algorithm

- In an operating system that uses paging, **a page replacement algorithm is needed** when a page fault occurs and no free page frame is available.

- In this case, one of the existing pages in memory must be replaced with the new page.

1. First In First Out (FIFO)
2. Optimal Page replacement
3. Least Recently Used (LRU)

Page Replacement Algorithm - First In First Out (FIFO)

- This is the simplest page replacement algorithm.

- In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue.

- When a page needs to be replaced page in the front of the queue is selected for removal.

Page Replacement Algorithm - First In First Out (FIFO)

- **Example 1:** Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3-page frames. Find the number of page faults using FIFO Page Replacement Algorithm.

Page reference 1, 3, 0, 3, 5, 6, 3

1	3	0	3	5	6	3
		0	0	0	0	3
	3	3	3	3	6	6
1	1	1	1	5	5	5
Miss	Miss	Miss	Hit	Miss	Miss	Miss

Total Page Fault = 6

Page Replacement Algorithm - First In First Out (FIFO)

- Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots ---> **3 Page Faults.**
- When 3 comes, it is already in memory so ---> 0 **Page Faults.**
- Then 5 comes, it is not available in memory, so it replaces the oldest page slot i.e 1. ---> 1 **Page Fault.**
- 6 comes, it is also not available in memory, so it replaces the oldest page slot i.e 3 ---> 1 **Page Fault.**
- Finally, when 3 come it is not available, so it replaces 0 1-page **fault.**

Page Replacement Algorithm - Optimal Page Replacement

- In this algorithm, **pages are replaced** which would not be used for the longest duration of time in the future.

- **Example:** Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find number of page fault using Optimal Page Replacement Algorithm.

Page Replacement Algorithm - Optimal Page Replacement

Page reference 7,0,1,2,0,3,0,4,2,3,0,3,2,3 No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

- **Total Page Fault = 6**

Page Replacement Algorithm - Least Recently Used

- In this algorithm, page will be replaced which is least recently used.

- **Example:** Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find number of page fault using Least Recently Used (LRU) Replacement Algorithm.

Page Replacement Algorithm - Least Recently Used

Page reference 7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Page Replacement Algorithm

- Consider page sequence 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2 and discuss working of following page replacement policies Also count page faults.

(use no. of Frames = 3)

i) FIFO

ii) LRU

Thank You